

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №6.

Тема: «АТД. Контейнеры»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

1. Определить класс-контейнер.
2. Реализовать конструкторы, деструктор, операции ввода-вывода, операцию присваивания.
3. Перегрузить операции, указанные в варианте.
4. Реализовать класс-итератор. С его помощью реализовать операции последовательного доступа.
5. Написать тестирующую программу, которая иллюстрирует выполнение всех операций.

Класс-контейнер: МНОЖЕСТВО (SET) с элементами типа `int`.

Перегружаемые операции:

- `[]` – доступ к элементу по индексу;
- `()` – определение размера множества (вектора);
- `--` – разность множеств;
- `--` – переход к предыдущему элементу (с помощью класса-итератора).

Анализ задачи

1. Создаём абстрактный класс `Triad` с полями `first`, `second`, `third` и чисто виртуальными методами `increment`, `print`.
2. Создаём класс `Date` - наследник `Triad`. Добавляем поля `year`, `month`, `day`. Переопределяем `increment` (увеличиваем `day` на 1, нормализуем дату). Добавляем метод `addDays` (увеличиваем `day` на `n`, нормализуем).
3. Создаём класс `Vector` с массивом указателей `Triad`. Реализуем метод `add` для добавления объектов, метод `incrementAll` для вызова `increment` у всех элементов, перегрузку `operator<<` для вывода.
4. В `main`: создаём `Vector`, добавляем в него объекты `Date`. Вызываем `incrementAll` - все даты увеличиваются на 1 день. Создаём указатель `Triad*` на объект `Date`, вызываем `increment` (полиморфизм). Через `dynamic_cast` приводим к `Date*` и вызываем `addDays`.

Код

```
#include <iostream>
#include <stdexcept>
using namespace std;

// Итератор
class SetIterator {
private:
    int* ptr;
public:
    SetIterator(int* p = nullptr) : ptr(p) {}

    SetIterator& operator--() { --ptr; return *this; }
    SetIterator operator--(int) { SetIterator temp = *this; --ptr; return temp; }
    SetIterator& operator++() { ++ptr; return *this; }
    bool operator!=(const SetIterator& other) const { return ptr != other.ptr; }
    int& operator*() { return *ptr; }
};

// Множество
class Set {
private:
    int* elements;
    int size;
```

```

int capacity;

bool contains(int value) const {
    for (int i = 0; i < size; i++)
        if (elements[i] == value) return true;
    return false;
}

public:
Set() : size(0), capacity(10) {
    elements = new int[capacity];
}

Set(const Set& other) : size(other.size), capacity(other.capacity) {
    elements = new int[capacity];
    for (int i = 0; i < size; i++) elements[i] = other.elements[i];
}

~Set() { delete[] elements; }

Set& operator=(const Set& other) {
    if (this != &other) {
        delete[] elements;
        size = other.size;
        capacity = other.capacity;
        elements = new int[capacity];
        for (int i = 0; i < size; i++) elements[i] = other.elements[i];
    }
    return *this;
}

int operator()() const { return size; }

int& operator[](int index) {
    if (index < 0 || index >= size) throw out_of_range("Индекс вне границ");
    return elements[index];
}

Set operator-(const Set& other) const {
    Set result;
    for (int i = 0; i < size; i++)
        if (!other.contains(elements[i]))
            result.add(elements[i]);
    return result;
}

void add(int value) {
    if (!contains(value)) {
        if (size >= capacity) {
            capacity *= 2;
            int* newElements = new int[capacity];
            for (int i = 0; i < size; i++) newElements[i] = elements[i];
            delete[] elements;
            elements = newElements;
        }
        elements[size++] = value;
    }
}

SetIterator begin() { return SetIterator(elements); }
SetIterator end() { return SetIterator(elements + size); }

friend istream& operator>>(istream& is, Set& s) {
    int n, val;
    cout << "Кол-во элементов: ";
    is >> n;
    for (int i = 0; i < n; i++) {
        cout << "Эл " << i + 1 << ": ";
        is >> val;
    }
}

```

```

        s.add(val);
    }
    return is;
}

friend ostream& operator<<(ostream& os, const Set& s) {
    os << "{ ";
    for (int i = 0; i < s.size; i++) {
        os << s.elements[i];
        if (i < s.size - 1) os << ", ";
    }
    os << " }";
    return os;
}
};

int main() {
    setlocale(LC_ALL, "Russian");

    Set A, B;

    cout << "Множество A:\n"; cin >> A;
    cout << "Множество B:\n"; cin >> B;

    cout << "\nA = " << A << endl;
    cout << "B = " << B << endl;
    cout << "|A| = " << A() << endl;
    cout << "|B| = " << B() << endl;

    if (A() > 0) cout << "A[0] = " << A[0] << endl;

    Set C = A - B;
    cout << "A - B = " << C << endl;

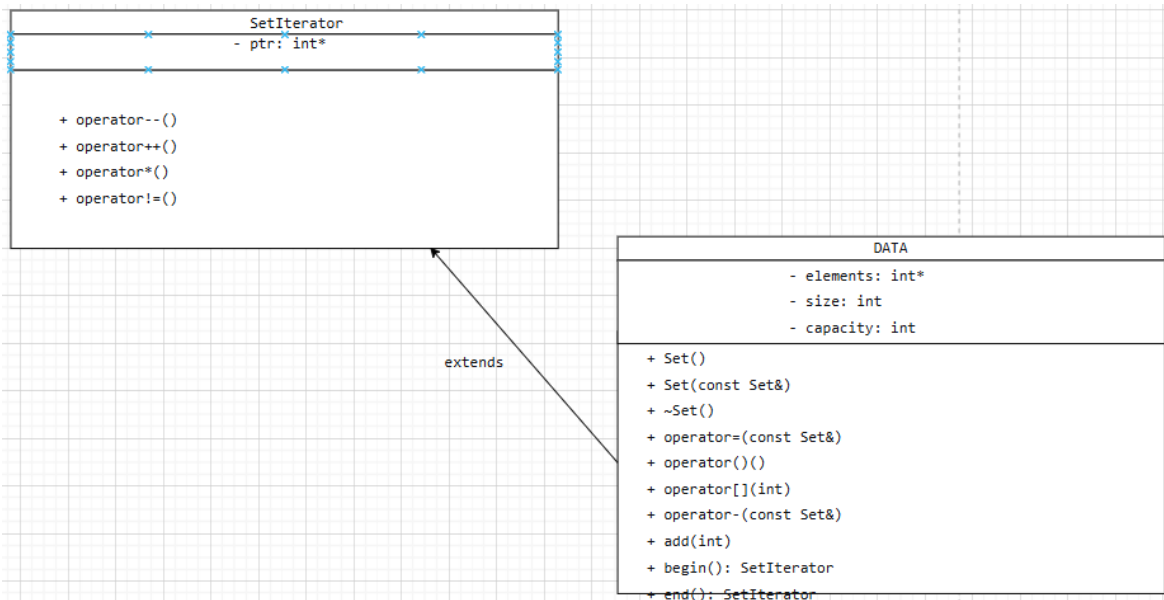
    cout << "\nИтератор по A: ";
    for (SetIterator it = A.begin(); it != A.end(); ++it)
        cout << *it << " ";

    if (A() > 0) {
        SetIterator it = A.end();
        --it;
        cout << "\nПоследний элемент A: " << *it;
    }

    cout << "\n\nГотово!" << endl;
    return 0;
}

```

UML диаграмма



Вывод программы

```

Множество A:
Кол-во элементов: 3
Эл 1: 3
Эл 2: 4
Эл 3: 5
Множество B:
Кол-во элементов: 4
Эл 1: 5
Эл 2: 3
Эл 3: 1
Эл 4: 6

A = { 3, 4, 5 }
B = { 5, 3, 1, 6 }
|A| = 3
|B| = 4
A[0] = 3
A - B = { 4 }

Итератор по A: 3 4 5
Последний элемент A: 5

Готово!
  
```

Ответы на вопросы

1. Абстрактный тип данных (АТД) — это тип, определяемый поведением (операциями), а не внутренним устройством. Примеры АТД: стек, очередь, список, множество, ассоциативный массив.
2. Абстракция через параметризацию — создание общих типов или функций с параметрами (типами, константами). Примеры в C++: шаблоны классов (`vector<T>`), шаблоны функций (`sort<T>`), шаблоны с нетповыми параметрами (`array<T, N>`).
3. Абстракция через спецификацию — описание интерфейса без указания реализации. Примеры в C++: абстрактные классы с чисто виртуальными методами, концепты (C++20), заголовочные файлы с объявлениями функций.

4. Контейнер — объект, хранящий другие объекты и предоставляющий доступ к ним. Примеры в C++: `vector`, `list`, `map`, `set`, `queue`, `stack`, `array`.

5. Группы операций в контейнерах: создание/уничтожение, добавление элементов, удаление элементов, доступ к элементам, проверка размера, итерация по элементам.

6. Виды доступа к элементам контейнера: прямой доступ по индексу (`vector`), последовательный доступ с помощью итераторов (`list`), доступ по ключу (`map`). Примеры: `mas[5]`, `it++`, `map["key"]`.

7. Итератор — объект, указывающий на элемент контейнера, позволяющий перемещаться по контейнеру и получать доступ к элементу. Аналог обобщённого указателя.

8. Итератор может быть реализован как класс с перегруженными операторами (`++`, `*`, `==`, `!=`). Внутри может содержать индекс (для массива) или указатель на узел (для списка).

9. Объединение контейнеров можно организовать через алгоритмы (`merge`, `set_union`), через вставку диапазона (`insert`, `copy`), через конкатенацию (для строк), или с помощью обёрток (`view` в C++20).

10. Контейнер из элементов ключ-значение предоставляет доступ по ключу (ассоциативный доступ). В C++ это `map` и `unordered_map`.

11. Контейнер, в котором вставка и удаление выполняются на одном конце, называется стек (`stack`).

12. Контейнером является: `d. int mas[100];` (массив хранит несколько элементов).

13. Не является контейнером: `d. int mas;` (одиночная переменная).

14. Доступ к элементам контейнера, реализованного как динамический массив с операцией доступа по индексу, будет прямым (произвольный доступ, $O(1)$).

15. Контейнер, реализованный как линейный список, обеспечивает последовательный доступ (только перебором от начала или конца, без доступа по индексу за $O(1)$).